

Universidade Federal
de Campina Grande



UNIVERSIDADE FEDERAL DE CAMPINA GRANDE
DEPARTAMENTO DE ENGENHARIA ELÉTRICA E INFORMÁTICA
PROGRAMA DE RESIDÊNCIA EM MICROELETRÔNICA
MCTI/Softex/IRede

RELATÓRIO TÉCNICO
Semana 2 - Construção dos RTL

Aluno: Guilherme José Mendes Jerônimo
Professor: Prof. Henrique Cunha
Semestre: 2026.2

Campina Grande - PB
2026

Sumário

1 Descrição dos Módulos da Cifragem e Decifragem AES **1**

1.1 Módulo minisoc_aes_sbox 1

1.2 Módulo minisoc_aes_sub_bytes 2

1.3 Módulo minisoc_aes_shift_rows 3

1.4 Módulo minisoc_aes_inv_sbox 4

1.5 Módulo minisoc_aes_inv_sub_bytes 5

1.6 Módulo minisoc_aes_inv_shift_rows 6

1 Descrição dos Módulos da Cifragem e Decifragem AES

1.1 Módulo minisoc_aes_sbox

Listing 1: Implementação da S-box Direta (Encriptação)

```
// S-box AES (FIPS-197, Figura 7). Puramente combinacional.
module minisoc_aes_sbox (
    input  logic [7:0] entrada,
    output logic [7:0] saida
);
  always_comb begin
    case (entrada)
      8'h00: saida = 8'h63;
      8'h01: saida = 8'h7c;
      8'h02: saida = 8'h77;
      // ... (mapeamento completo de 256 posicoes: 8'h03 a 8'hfe)
      8'hff: saida = 8'h16;
    endcase
  end
endmodule

```

O módulo `minisoc_aes_sbox` implementa a substituição não linear direta de bytes utilizada na etapa de encriptação do AES (especificada na FIPS-197, Figura 7)[cite: 4]. Matematicamente, a operação realiza a busca pelo inverso multiplicativo do byte no campo finito Galois $GF(2^8)$, seguida de uma transformação afim[cite: 4].

Em hardware, essa estrutura é sintetizada como uma Tabela de Consulta (*Look-Up Table* - LUT) totalmente combinacional de 256 posições[cite: 4]. O circuito possui latência zero e não depende de relógio[cite: 4]. A Tabela 1 apresenta a matriz hexadecimal completa dessa transformação.

Tabela 1: Matriz de mapeamento hexadecimal da S-box direta (FIPS-197)

y \ x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1_	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2_	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3_	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4_	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5_	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6_	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7_	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8_	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9_	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
A_	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
B_	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
C_	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
D_	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
E_	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
F_	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

1.2 Módulo minisoc_aes_sub_bytes

Listing 2: Implementação do SubBytes (Encriptação)

```
// SubBytes: 16 instancias da S-box. Byte 0 em [127:120], byte 15 em [7:0].
module minisoc_aes_sub_bytes (
    input  logic [127:0] i_state,
    output logic [127:0] o_state
);
  for (genvar n = 0; n < 16; n++)
  begin
    minisoc_aes_sbox u_sbox (
        .entrada(i_state[127-8*n -: 8]),
        .saida  (o_state[127-8*n -: 8])
    );
  end
endmodule

```

O módulo `minisoc_aes_sub_bytes` aplica a substituição S-box paralelamente sobre todos os 16 bytes do vetor de estado de 128 bits (`i_state`)[cite: 6]. Ele instancia 16 cópias do bloco `minisoc_aes_sbox` através da diretiva `genvar`[cite: 6].

A indexação utiliza a sintaxe de corte indexado (`-: 8`), dividindo os 128 bits em 16 blocos de 8 bits[cite: 6]: o byte 0 localiza-se nos bits `[127:120]` e o byte 15 nos bits

`[7:0]`[cite: 6]. A operação é realizada em um único ciclo combinacional[cite: 6].

1.3 Módulo minisoc_aes_shift_rows

Listing 3: Implementação do ShiftRows (Encriptação)

```
// ShiftRows: so reordenacao de fios. Byte n = s[r][c], r = n%4, c = n/4; s'[r
    ][c] = s[r][(c+r) mod 4].
module minisoc_aes_shift_rows (
    input  logic [127:0] entrada,
    output logic [127:0] saida
);
  for (genvar n = 0; n < 16; n++) begin : g_sr
    localparam int R   = n % 4;
    localparam int C   = n / 4;
    localparam int SRC = R + 4 * ((C + R) % 4);
    assign saida[127-8*n -: 8] = entrada[127-8*SRC -: 8];
  end
endmodule

```

O módulo `minisoc_aes_shift_rows` executa a rotação circular dos bytes nas linhas da matriz de estado do AES durante a encriptação[cite: 5]. A Linha 0 não sofre deslocamento, a Linha 1 rotaciona 1 posição à esquerda, a Linha 2 rotaciona 2 posições e a Linha 3 rotaciona 3 posições[cite: 5].

A equação $SRC = R + 4 \times ((C + R) \bmod 4)$ mapeia o byte de origem[cite: 5]. Esse circuito é puramente um permutador de fios e não consome recursos lógicos na FPGA/ASIC[cite: 5]. A Tabela 2 lista os 16 casos do laço.

Tabela 2: Mapeamento de bytes do ShiftRows (Encriptação)

n	Linha ($R = n \bmod 4$)	Coluna ($C = \lfloor n/4 \rfloor$)	Origem (SRC)	Mapeamento de Byte
0	0	0	0	`saida[0]` ← `entrada[0]`
1	1	0	5	`saida[1]` ← `entrada[5]`
2	2	0	10	`saida[2]` ← `entrada[10]`
3	3	0	15	`saida[3]` ← `entrada[15]`
4	0	1	4	`saida[4]` ← `entrada[4]`
5	1	1	9	`saida[5]` ← `entrada[9]`
6	2	1	14	`saida[6]` ← `entrada[14]`
7	3	1	3	`saida[7]` ← `entrada[3]`
8	0	2	8	`saida[8]` ← `entrada[8]`
9	1	2	13	`saida[9]` ← `entrada[13]`
10	2	2	2	`saida[10]` ← `entrada[2]`
11	3	2	7	`saida[11]` ← `entrada[7]`
12	0	3	12	`saida[12]` ← `entrada[12]`
13	1	3	1	`saida[13]` ← `entrada[1]`
14	2	3	6	`saida[14]` ← `entrada[6]`
15	3	3	11	`saida[15]` ← `entrada[11]`

1.4 Módulo minisoc_aes_inv_sbox

Listing 4: Implementação da S-box Inversa (Decifragem)

```
// InvS-box AES (FIPS-197, Figura 14). Puramente combinacional.
module minisoc_aes_inv_sbox (
    input  logic [7:0] entrada,
    output logic [7:0] saida
);
  always_comb begin
    case (entrada)
      8'h00: saida = 8'h52;
      8'h01: saida = 8'h09;
      8'h02: saida = 8'h6a;
      // ... (mapeamento completo de 256 posicoes: 8'h03 a 8'hfe)
      8'hff: saida = 8'h7d;
    endcase
  end
endmodule

```

O módulo `minisoc_aes_inv_sbox` executa a substituição inversa de bytes para a etapa de decifragem (FIPS-197, Figura 14)[cite: 1]. A operação desfaz a transformação afim e calcula o inverso multiplicativo correspondente em $GF(2^8)$[cite: 1].

A implementação é puramente combinacional, estruturada via bloco `case` de 256 entradas[cite: 1]. A Tabela 3 apresenta a matriz hexadecimal completa da S-box inversa.

Tabela 3: Matriz de mapeamento hexadecimal da InvS-box (FIPS-197)

y \ x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
1_	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
2_	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
3_	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
4_	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
5_	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
6_	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
7_	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
8_	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
9_	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
A_	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
B_	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
C_	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
D_	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
E_	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
F_	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

1.5 Módulo minisoc_aes_inv_sub_bytes

Listing 5: Implementação do InvSubBytes (Decifragem)

```
module minisoc_aes_inv_sub_bytes (
    input  logic [127:0] entrada,
    output logic [127:0] saida
);
  for (genvar n = 0; n < 16; n++)
    begin
      minisoc_aes_inv_sbox u_isbox (
          .entrada(entrada[127-8*n -: 8]),
          .saida(saida[127-8*n -: 8])
      );
    end
endmodule

```

O módulo `minisoc_aes_inv_sub_bytes` aplica a substituição S-box inversa sobre os 16 bytes do vetor de estado de 128 bits durante a decodificação[cite: 3]. O laço `genvar` instancia 16 módulos `minisoc_aes_inv_sbox`[cite: 3].

Cada byte da palavra de entrada de 128 bits é processado em paralelo[cite: 3], garantindo latência zero e operação estritamente combinacional[cite: 3].

1.6 Módulo minisoc_aes_inv_shift_rows

Listing 6: Implementação do InvShiftRows (Decifragem)

```
// InvShiftRows: linha r rotaciona a direita r posicoes.
module minisoc_aes_inv_shift_rows (
    input  logic [127:0] entrada,
    output logic [127:0] saida
);
  for (genvar n = 0; n < 16; n++)
    begin
      localparam int R      = n % 4;
      localparam int C      = n / 4;
      localparam int result = R + 4 * ((C + 4 - R) % 4);
      assign saida[127-8*n -: 8] = entrada[127-8*result -: 8];
    end
endmodule

```

O módulo `minisoc_aes_inv_shift_rows` executa a rotação inversa das linhas da matriz de estado durante a decifragem[cite: 2]. A rotação é feita para a direita: a Linha 0 desloca 0 posições, a Linha 1 desloca 1, a Linha 2 desloca 2 e a Linha 3 desloca 3[cite: 2].

A fórmula $result = R + 4 \times ((C + 4 - R) \bmod 4)$ calcula o byte de origem[cite: 2]. A Tabela 4 detalha os 16 casos do laço e a Tabela 5 mostra a distribuição dos bits na matriz de estado.

Tabela 4: Mapeamento dos índices do InvShiftRows

n	Linha ($R = n \bmod 4$)	Coluna ($C = \lfloor n/4 \rfloor$)	Origem ($result$)	Mapeamento de Byte
0	0	0	0	`saida[0]` ← `entrada[0]`
1	1	0	13	`saida[1]` ← `entrada[13]`
2	2	0	10	`saida[2]` ← `entrada[10]`
3	3	0	7	`saida[3]` ← `entrada[7]`
4	0	1	4	`saida[4]` ← `entrada[4]`
5	1	1	1	`saida[5]` ← `entrada[1]`
6	2	1	14	`saida[6]` ← `entrada[14]`
7	3	1	11	`saida[7]` ← `entrada[11]`
8	0	2	8	`saida[8]` ← `entrada[8]`
9	1	2	5	`saida[9]` ← `entrada[5]`
10	2	2	2	`saida[10]` ← `entrada[2]`
11	3	2	15	`saida[11]` ← `entrada[15]`
12	0	3	12	`saida[12]` ← `entrada[12]`
13	1	3	9	`saida[13]` ← `entrada[9]`
14	2	3	6	`saida[14]` ← `entrada[6]`
15	3	3	3	`saida[15]` ← `entrada[3]`

Tabela 5: Mapeamento dos índices dos bits na matriz de estado do AES

	Coluna 0	Coluna 1	Coluna 2	Coluna 3
Linha 0	[127:120]	[95:88]	[63:56]	[31:24]
Linha 1	[119:112]	[87:80]	[55:48]	[23:16]
Linha 2	[111:104]	[79:72]	[47:40]	[15:8]
Linha 3	[103:96]	[71:64]	[39:32]	[7:0]